

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Full Stack development MERN Practical

M2 Practical-9

Roll No.:T017	Name: Radhika Mali
Class:TYIT	Batch:1
Date of Assignment: 30-9-2026	Date/Time of Submission: 30-9-2026

9.User Data Management

- a. Create an API to add user details using Express and MongoDB.
- b. Design a React form to send user data to the backend.
- c. Retrieve and display user data from MongoDB.
- d. Implement an update operation for existing records.
- e. Implement the delete operation for selected records.
- f. Display updated records dynamically in React.

Steps:

1. Create mongo database UserDB -> collection name users
2. Find the insert all data.

Code:

```
db.users.insertMany([
  {
    name: "Radhika",
    email: "radhika@gmail.com",
    city: "Mumbai"
  },
  {
    name: "Abhi",
    email: "abhi@gmail.com",
    city: "Pune"
  },
  {
    name: "DH",
    email: "dh@gmail.com",
    city: "UP"
  }
])
```

```
> db.users.find()
< {
  _id: ObjectId('6abcbd32553bf3f05b75f26d'),
  name: 'Radhika',
  email: 'radhika@gmail.com',
  city: 'Mumbai'
}
{
  _id: ObjectId('6abcbd32553bf3f05b75f26e'),
  name: 'Abhi',
  email: 'abhi@gmail.com',
  city: 'Pune'
}
{
  _id: ObjectId('6abcbd32553bf3f05b75f26f'),
  name: 'DH',
  email: 'dh@gmail.com',
  city: 'UP'
}
```

Step 2 - Create backend

- 1) Create a folder practical9 & open that in vs code
- 2) Press ctrl+~ to open terminal & follow next step
- 3) Run -> npm init -y to create package.json file.
- 4) Add server.js file
- 5) Run command -> npm install mongoose express cors
- 6) Click + icon(new file) near practical9 & name it server.js
- 7) Add following code into server.js file
const express = require("express");

Code:

```

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
const PORT = 3000;
// Middleware
app.use(cors());
app.use(express.json());
// MongoDB Connection
mongoose.connect("mongodb://127.0.0.1:27017/UserDB")
.then(() => {
  console.log("MongoDB Connected Successfully");
})
.catch((error) => {
  console.log("MongoDB Connection Failed");
  console.log(error);
});
// GET - Retrieve Users
app.get("/users", async (req, res) => {
  try {
    const users = await mongoose.connection.db
      .collection("users")
      .find()
      .toArray();
    res.json(users);
  }
  catch (error) {
    res.status(500).json({ message: "Error fetching users" });
  }
});
// POST - Add User
app.post("/users", async (req, res) => {
  try {
    const user = req.body;
    await mongoose.connection.db
      .collection("users")
      .insertOne(user);
    res.json({ message: "User added successfully" });
  }
  catch (error) {
    res.status(500).json({ message: "Error adding user" });
  }
});
// PUT - Update User
app.put("/users/:id", async (req, res) => {
  try {
    const id = new mongoose.Types.ObjectId(req.params.id);
    const updatedUser = req.body;
    await mongoose.connection.db
      .collection("users")
      .updateOne({ _id: id },
        {
          $set: {
            name: updatedUser.name,
            email: updatedUser.email,
            city: updatedUser.city
          }
        }
      );
    res.json({ message: "User updated successfully" });
  }

```

```

}
catch (error) {
  res.status(500).json({ message: "Error updating user" });
}
});
// DELETE - Delete User
app.delete("/users/:id", async (req, res) => {
  try {
    const id = new mongoose.Types.ObjectId(req.params.id);
    await mongoose.connection.db
      .collection("users")
      .deleteOne({
        _id: id
      });
    res.json({ message: "User deleted successfully" });
  }
  catch (error) {
    res.status(500).json({ message: "Error deleting user" });
  }
});
// Start Server
app.listen(PORT, () => {
  console.log(`Server is running on
  http://localhost:${PORT}`);
});

```

Step 3 - Create frontend

- 1) In terminal create react project type `npm create vite@latest frontend`
- 2) Choose react, javascript, ESLint
- 3) In terminal move to frontend & install axios
`npm install axios`
- 4) expand src & replace App.jsx code with following

Code:

```

import { useEffect, useState } from "react";
import axios from "axios";
function App() {
  // Store all users
  const [users, setUsers] = useState([]);
  // Form data
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [city, setCity] = useState("");
  // Store ID of user being edited
  const [editId, setEditId] = useState(null);
  // Get Users from MongoDB
  const getUsers = () => {
    axios.get("http://localhost:3000/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.log(error);
      });
  };
  // Load users when page starts
  useEffect(() => { getUsers(); }, []);
  // Add or Update User
  const saveUser = () => {

```

```

const user = {
  name: name,
  email: email,
  city: city
};
// UPDATE existing user
if (editId) {
  axios.put(`http://localhost:3000/users/${editId}`, user)
    .then((response) => {
      alert(response.data.message);
      clearForm();
      getUsers();
    })
    .catch((error) => { console.log(error); });
}
// ADD new user
else {
  axios.post("http://localhost:3000/users", user)
    .then((response) => {
      alert(response.data.message);
      clearForm();
      getUsers();
    })
    .catch((error) => { console.log(error); });
}
// Edit User
const editUser = (user) => {
  setEditId(user._id);
  setName(user.name);
  setEmail(user.email);
  setCity(user.city);
};
// Delete User
const deleteUser = (id) => {
  if (window.confirm("Are you sure you want to delete
this user?")) {
    axios.delete(`http://localhost:3000/users/${id}`)
      .then((response) => {
        alert(response.data.message);
        getUsers();
      })
      .catch((error) => { console.log(error); });
  }
};
// Clear Form
const clearForm = () => {
  setName("");
  setEmail("");
  setCity("");
  setEditId(null);
};
return (
  <div>
    <h1>User Management System</h1>
    { /* User Form */ }
    <h2>
      {editId ? "Update User" : "Add User"}
    </h2>
    <input type="text"
      placeholder="Enter Name"
      value={name}
      onChange={(e) => setName(e.target.value)}
    />
    <br />
    <input type="email"
      placeholder="Enter Email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
    />
    <br />
    <input type="text"
      placeholder="Enter City"
      value={city}
      onChange={(e) => setCity(e.target.value)}
    />
    <br />
    <button onClick={saveUser}>
      {editId ? "Update User" : "Add User"}
    </button>
    {editId && (<button onClick={clearForm}>
      Cancel
    </button>)}
  </hr />
  { /* Display Users */ }
  <h2>User List</h2>
  <table border="1">
    <thead>
      <tr>
        <th>Name</th>
        <th>Email</th>
        <th>City</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody>
      {users.map((user) => (
        <tr key={user._id}>
          <td>{user.name}</td>
          <td>{user.email}</td>
          <td>{user.city}</td>
          <td>
            <button onClick={() => editUser(user)}>
              Edit
            </button>
            <button onClick={() => deleteUser(user._id)}>
              Delete
            </button>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
);
}
export default App;

```

Step 4 - Execution part

- 1) Split terminal
- 2) execute backend(server) means node server.js
- 3) execute frontend using npm run dev

Output:

User Management System

Add User

Name	Email	City	Actions	
Radhika	radhika@gmail.com	Mumbai	Edit	Delete
Abhi	abhi@gmail.com	Pune	Edit	Delete
DH	dh@gmail.com	UP	Edit	Delete

localhost:5173 says

User added successfully

Name	Email	City	Actions	
Radhika	radhika@gmail.com	Mumbai	Edit	Delete
Abhi	abhi@gmail.com	Pune	Edit	Delete
DH	dh@gmail.com	UP	Edit	Delete
asmitha	asmi@gmail.com	Aarey	Edit	Delete